

Nix for science and fun

Alán F. Muñoz and Ankur Kumar

Imaging Platform, Broad Institute of Harvard and MIT



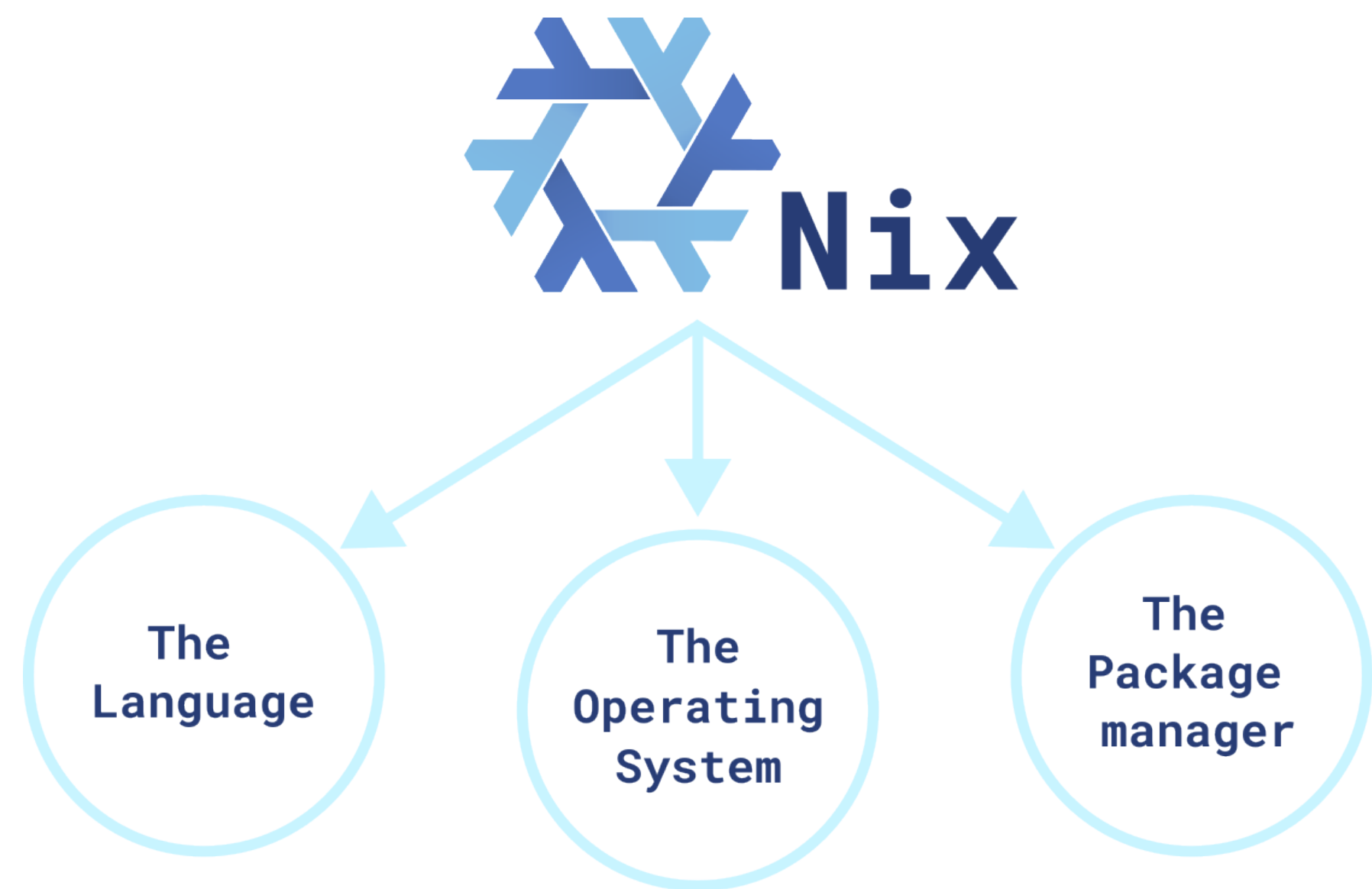
Abstract

One of the biggest challenges for the biological sciences is the replication crisis we face. In addition to inherent biological noise and ever-growing datasets, limited focus on reproducibility of computational processes and their associated results hinders novel discoveries. Improving the reliability and efficiency of scientific research will increase the credibility of current and future scientific literature and accelerate discovery. While there have been improvements in recent years, current technologies fall short of principally solving the problem of software reproducibility. Nix is a framework for software deployment that warrants correct resolution of dependency trees to provide fully reproducible and deterministic packages and environments.

What is Nix?

I am unable to re-run your analysis to verify the reported results.

- Reviewer 2



Problem: Dependency resolution

Solution:

Most package managers and workflow tools are not theoretically sound to provide reproducibility guarantees.

```
# Package management with APT on Ubuntu
# Install git system wide
apt install git
```

```
# Install CellProfiler via pip
pip install cellprofiler
```

- Packages are installed system wide
- Version specification is only possible for the top level of the dependency tree
- Building packages from source is usually not supported or not encouraged

```
# Create temporary shell
# with nixpkgs' git
nix shell nixpkgs#git
```

```
# Runs cellprofiler
nix run github:Cellprofiler/
Cellprofiler#cellprofiler
```

- Packages are always isolated and cannot be accessed outside the temporary shell
- Version specification is possible of the entire dependency tree
- Packages can be built from source and have first class support

Problem: Dependency conflicts

Solution:

```
# Install pytorch with cuda 11
apt install cuda-toolkit-11-8
```

```
# Install pytorch with cuda 12
apt install cuda-toolkit-12-8
```

- CUDA is installed system wide
- Cannot install two versions of cuda at the same time without manual symlink and env vars management

```
# Create temporary shell with:
# cuda 11
nix shell nixpkgs#cudaPackages_11.cudatoolkit
```

```
# cuda 12
nix shell nixpkgs#cudaPackages_12.cudatoolkit
```

- Cuda is not installed system-wide
- Multiple cuda versions can coexist without further environment management

Problem: Environment replication

Solution:

```
conda env create -f env.yaml
```

System wide dependencies cannot be easily shared without containerization.

- Conda can manage some system libraries, but not all

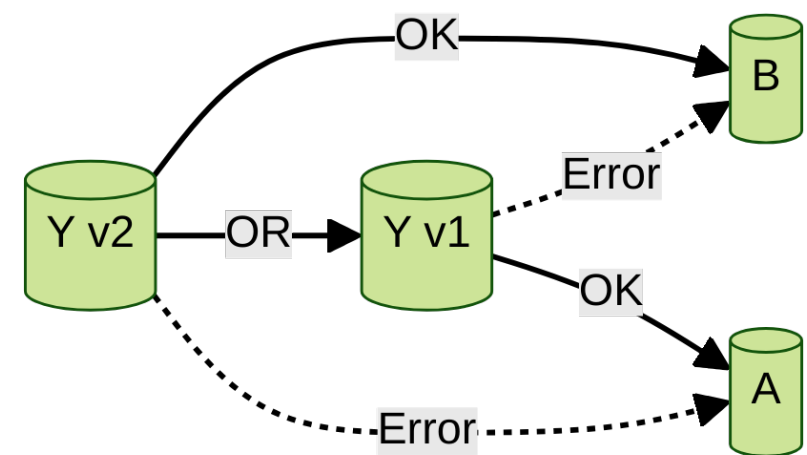
```
nixos-shell --flake github:leoank/neusis#copy
```

Problem: Projects with different requirements

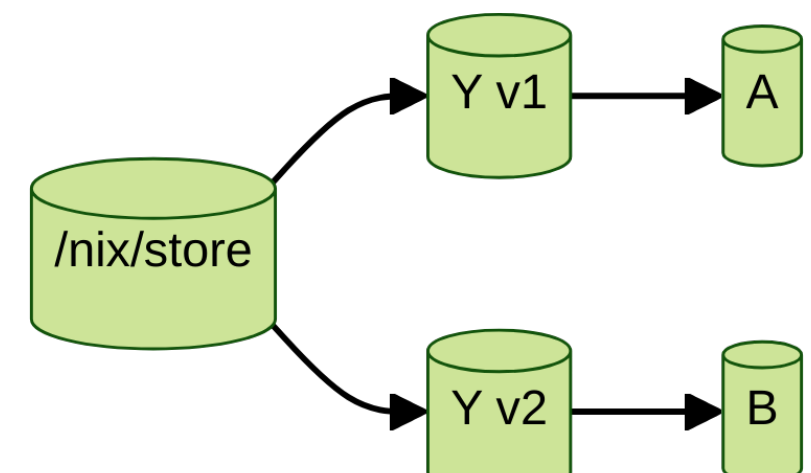
Solution:

On most operating systems, packages are installed system wide and are not isolated. This makes it very challenging to work with projects requiring older or newer version of dependencies without breaking the rest of the system.

Normal systems:

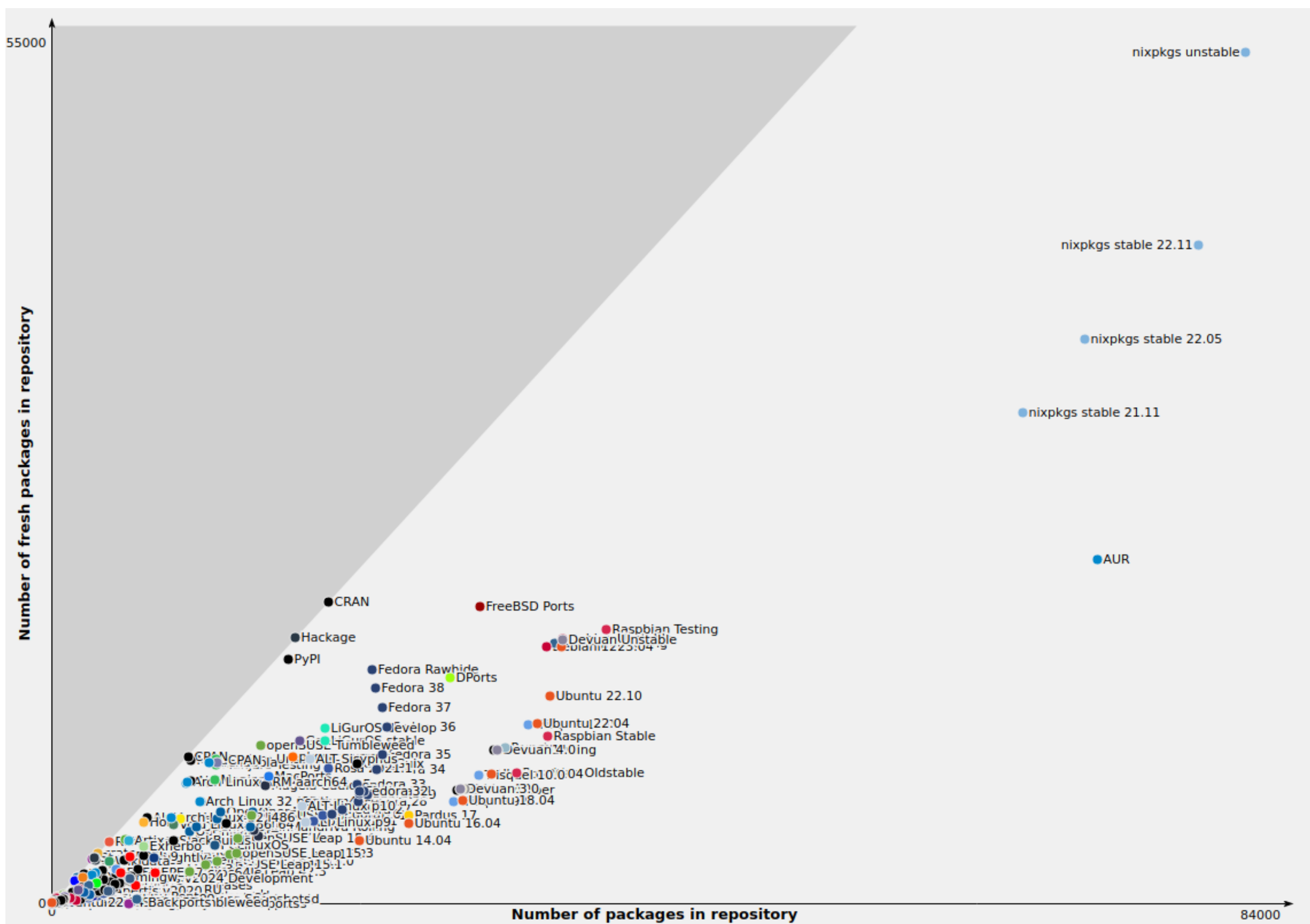


Nix:

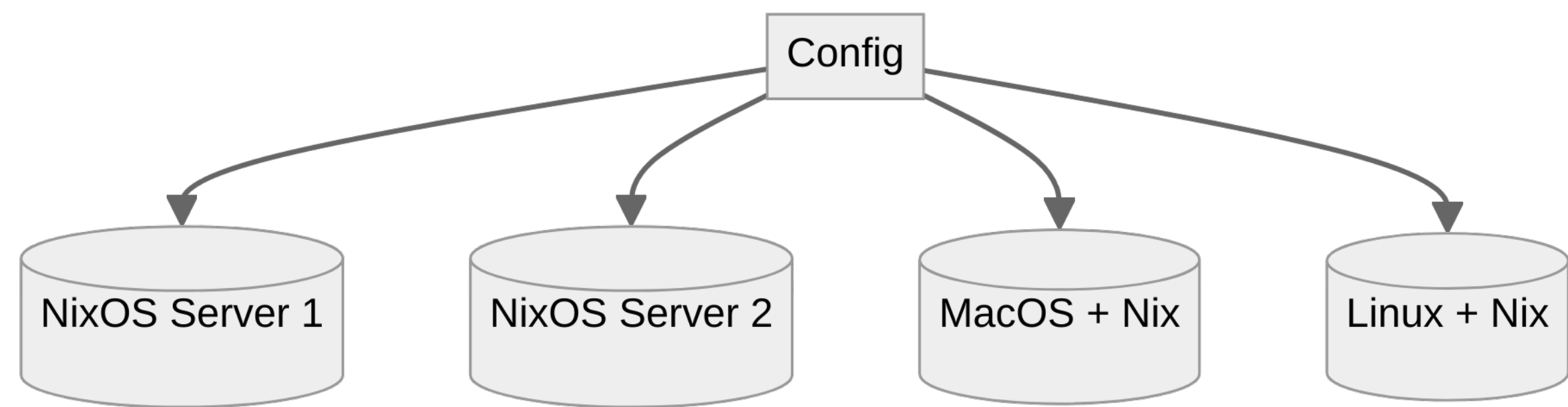


Benefits

100k+ packages as of April 2025



Consistent environments across different machines and architectures



Reliably package and deploy software

```
# file.nix
{inputs}:
{
  output = buildPackage{
    unpackPhase = {...};
    buildInputs = {...};
    buildPhase = {...};
    installPhase = {...};
  };
}

# $ nix build file.nix
# -> /nix/store/is2vyu9sxn4w9rrgqwg9ghlcf3slfv34-emacs-30.1.drv
# -> /nix/store/is2vyu9sxn4w9rrgqwg9ghlcf3slfv34-emacs-30.1
```

Generate entire pipelines with multiple languages and frameworks

This is useful for updating figures and re-run entire multi-language pipelines becomes a non-issue.

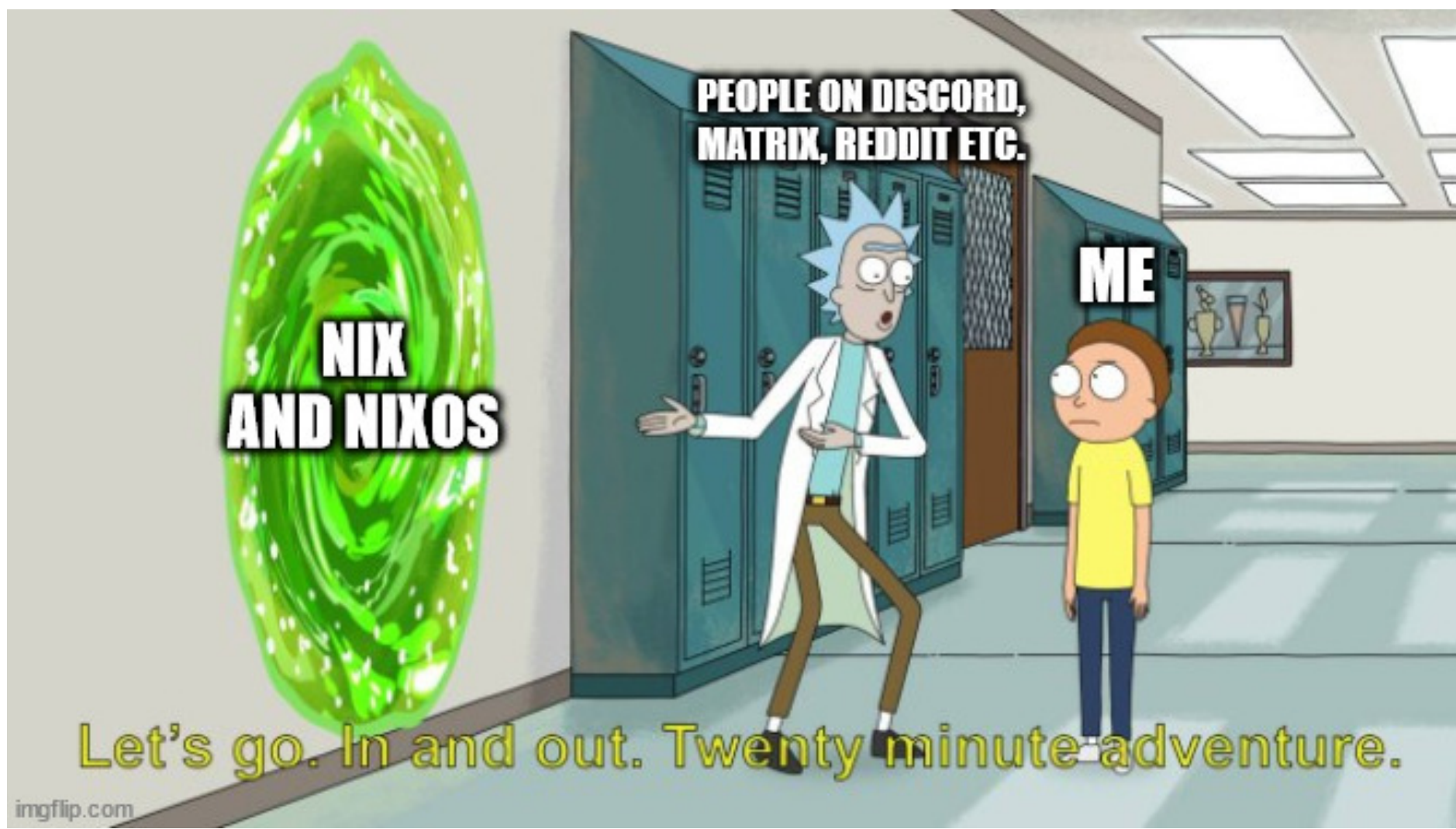
```
#!/usr/bin/env nix-shell
#! nix-shell -i bash --pure
#! nix-shell -I nixpkgs=https://github.com/NixOS/nixpkgs/archive/205a5.tar.gz
#! nix-shell -p bash wget awk R rPackages.Seurat uv
```

```
wget https://remote_data.com/files_of_interest.txt
```

```
awk -f preprocessing_script.awk files_of_interest.txt > preprocessed_data
```

```
Rscript script.R preprocessed_data > new_data
for pyscript in python_scripts*.py; do
  uv run $pyscript new_data
done
```

Demos



Run any package in a disposable environment

```
nix run nixpkgs#inkscape
nix run github:Cellprofiler/Cellprofiler#cellprofiler
export NIXPKGS_ALLOW_UNFREE=1 && nix run nixpkgs#vscode --impure
```

Ask us for a demonstration!

- Setting up a specific analysis environment
- Walk me through the CellProfiler derivation
- Replicate results from a published paper
- Run a VM configured using NixOS
- Set up your package of preference
- Override a package version deep in the dependency tree
- Set up a flake for a reproducible environment
- Help me set up Nix package manager on my system

Also reproduce your entire home environment in other computers, across Linux, Windows, MacOS and Android*

Additional resources and links

